# AI Study App

## Project Plan & Sprint Backlog

Agile Development Plan | 30-Day MVP

Flutter + FastAPI + Azure | gstack Methodology

---

Date: April 2026
Collaborators: Human + Claude Code
Target Market: United States

# 0. Builder Ethos

This project follows the gstack builder ethos. These principles are not aspirational guidelines — they are enforced by our Claude Code configuration, review agents, and skill definitions. Every sprint, every task, every commit is held to these standards.

**Boil the Lake**

AI-assisted coding makes the marginal cost of completeness near-zero. When the complete implementation costs minutes more than the shortcut — do the complete thing. Every time. Tests ship with code, not in follow-up PRs. Every screen handles loading, error, and empty states. Every API endpoint has full validation and error handling. "Ship the shortcut" is legacy thinking.

**Search Before Building**

The 1000x engineer's first instinct is "has someone already solved this?" not "let me design it from scratch." Three layers of knowledge: Layer 1 (tried and true), Layer 2 (new and popular — scrutinize what the crowd says), Layer 3 (first principles — the most valuable, prize above all else). The eureka moment is discovering why the conventional approach is wrong for your specific case.

**Anti-Slop Mandate**

No placeholder code. No generic variable names. No copy-paste without understanding. No "this should work" — verify it works. No screens that only handle the happy path. No TODO comments that defer real work. Every function earns its existence. Every line earns its place. The code reviewer agent blocks slop before it merges.

**Confusion Protocol**

When you encounter architectural ambiguity — STOP. Do not guess. Do not fill in assumptions. Ask the human. Wrong assumptions compound into wrong architecture. The cost of asking is one message. The cost of guessing wrong is a rewrite.

# 1. Project Overview

The AI Study App is a multi-tenant, AI-powered learning platform that generates personalized questions and flash cards from admin-uploaded study materials. The app targets two primary user segments in the US market: families (parent + child) and educational institutions (teacher + students). It uses adaptive learning algorithms to tailor content difficulty and topic selection to each student's evolving knowledge state.

## 1.1 Tech Stack

- Mobile Apps: Flutter 3.x / Dart (single codebase, Admin + Student via flavor system)
- State Management: Riverpod 2.x with code generation (@riverpod annotation)
- Navigation: GoRouter with typed route parameters
- HTTP Client: Dio with auth/error/logging interceptors
- Models: Freezed + json_serializable for immutable data classes
- Backend API: Python 3.12, FastAPI, hosted on Azure Container Apps
- Object Database: Azure Cosmos DB (MongoDB API)
- Vector Store: Azure AI Search
- AI Model: Hosted on Azure AI Foundry (GPT-4o or equivalent)
- AI Orchestration: Custom MCP Server (Python, containerized)
- Document Processing: Azure Document Intelligence
- Authentication: Azure AD B2C (Microsoft Entra External ID)
- Content Safety: Azure AI Content Safety
- Caching: Azure Cache for Redis
- Async Processing: Azure Functions + Azure Service Bus
- Push Notifications: Azure Notification Hubs + Firebase Cloud Messaging

## 1.2 Architecture Principles

The system follows a layered architecture with clear separation of concerns. The Flutter apps communicate exclusively with the public API gateway. The API gateway handles authentication, rate limiting, and routing. Behind the gateway, the backend services are split into three categories: synchronous request handlers (CRUD, question generation), async processors (document ingestion, status recalculation), and the AI pipeline (MCP server coordinating retrieval and generation). All services share Cosmos DB as the persistence layer but access it through different patterns, with the sync handlers using Redis-cached reads for hot paths and the async processors writing directly.

Multi-tenancy is enforced at the database level. Each tenant gets its own Cosmos DB database, and each workspace gets its own collection within that database. Cross-tenant data access is architecturally impossible, not just permission-gated. This provides both security isolation and performance isolation.

The Flutter app uses a feature-first folder structure with Riverpod for dependency injection and state management. Each feature module (auth, questions, flashcards, gamification, admin) is self-contained with its own data layer, domain models, and presentation layer. A single codebase serves both the Admin and Student experiences through a flavor system, where feature availability is gated by a flavor configuration injected at app startup.

## 1.3 Work Division Philosophy

Tasks are divided between two collaborators: the Human developer and Claude Code. The general principle is that the Human handles architecture decisions, Azure resource provisioning, complex UI interactions and animations, and integration testing. Claude Code handles boilerplate code generation, database schema implementation, API endpoint scaffolding, Flutter

feature module scaffolding, utility code, and test writing. Both collaborate on the AI pipeline, prompt engineering, the adaptive learning algorithm, and design-critical screens. Each sprint has clearly marked ownership per task.

The gstack sprint process governs how we work: Think (office-hours) → Plan (plan-ceo-review, plan-eng-review) → Build → Review (review) → Test (qa) → Ship (ship) → Reflect (retro). Every skill feeds into the next. Nothing falls through the cracks because every step knows what came before it.

## 2. Repository Structure

The project uses a monorepo structure with clear separation between the backend services, Flutter app, shared configuration, and infrastructure definitions.

**ai-study-app/**

- backend/ — FastAPI application, all Python backend code
- backend/app/api/ — API route handlers organized by domain (auth, tenants, workspaces, users, documents, questions, gamification, moderation, analytics)
- backend/app/models/ — Pydantic models for request/response validation and Cosmos DB document schemas
- backend/app/services/ — Business logic layer (adaptive learning engine, gamification engine, taxonomy manager, moderation service)
- backend/app/core/ — Configuration, database connections, auth middleware, rate limiting
- backend/app/workers/ — Async processing workers (document ingestion, status recalculation, taxonomy regeneration)
- backend/tests/ — Unit and integration tests (pytest)
- mcp-server/ — MCP server implementation for AI orchestration
- flutter_app/ — Flutter application (single codebase, two flavors)
- flutter_app/lib/core/ — Theme, constants, DI setup, routing, shared extensions
- flutter_app/lib/features/ — Feature modules (auth, home, questions, flashcards, gamification, admin)
- flutter_app/lib/shared/ — Shared widgets, models, services used across features
- flutter_app/test/ — Widget and unit tests
- flutter_app/integration_test/ — End-to-end integration tests
- infra/ — Bicep/Terraform templates for Azure resource provisioning
- docs/ — Architecture decision records, API specs, onboarding guides
- .claude/ — Claude Code configuration (settings, skills, agents, rules)

## 3. Cosmos DB Schema Design

Each tenant gets a dedicated database named tenant_{tenant_id}. Within each tenant database, the following collections exist. Each collection stores JSON documents with a consistent structure.

## 3.1 Collections Overview

### users Collection

Stores all user profiles regardless of role. Each document contains the user's Azure AD B2C object ID, their tenant-level role (tenant_admin or member), an array of workspace memberships with per-workspace roles (workspace_admin or student), personal profile information (display name, age/grade if configured, avatar), and account metadata (created_at, last_login, soft_deleted flag). The partition key is user_id.

### workspaces Collection

Stores workspace configuration. Each document contains the workspace name and description, settings (question frequency, moderation thresholds, gamification config including daily goal and leaderboard visibility), the dynamic taxonomy object (topic tree, dependency graph, per-topic metadata), milestone definitions set by the admin, and invite code configurations. The partition key is workspace_id.

### documents Collection

Stores metadata about uploaded documents. Each document record includes the original filename, file type, Azure Blob Storage URL for the raw file, processing status (queued, extracting, analyzing, vectorizing, ready, flagged, failed), extracted text reference, topic tags generated during ingestion, moderation results, and upload metadata (who uploaded, when, file size). The partition key is workspace_id.

### knowledge_states Collection

Stores each student's computed learning state per workspace. Each document contains per-topic mastery scores (0–100), recency scores, difficulty ceilings (easy/medium/hard), attempt counts and success rates at each difficulty level, streak data, and last-updated timestamps. This is the precomputed summary that the adaptive learning engine reads on every question generation. The partition key is user_id.

### interactions Collection

Append-only event log of every student interaction. Each event records the question or flash card ID, the student's response, correctness, difficulty, topic, time taken, timestamp, and the XP awarded. This is the source of truth from which knowledge_states are recomputed. The partition key is user_id with a composite key including timestamp for efficient range queries.

### gamification Collection

Stores gamification profiles per user per workspace. Each document tracks total XP, per-topic XP, current level, current streak count, longest streak, streak maintenance log (daily), earned badges with timestamps, and a daily activity summary. The partition key is user_id.

### moderation_log Collection

Audit trail for all moderation events. Each record includes the content that was scanned (reference, not full text), the source (upload or AI-generated), the Azure Content Safety verdict and severity scores, the resolution (auto-approved, auto-rejected, or admin-reviewed), and the admin who resolved it if applicable. The partition key is workspace_id.

### question_queue Collection

Stores pre-generated questions waiting to be served. When the system generates questions ahead of time (prefetching), they're stored here with their topic, difficulty, format, and expiry timestamp. The partition key is user_id.

# 4. Sprint Plan (6 Sprints, 30 Days)

The project is divided into six sprints of five days each. Sprints 1–2 focus on foundational infrastructure. Sprints 3–4 build the core learning features. Sprints 5–6 add gamification, polish, and prepare for launch. Each sprint has a clear goal, a set of tasks with ownership and priority, and definition-of-done criteria. The gstack sprint cycle applies: Think → Plan → Build → Review → Test → Ship → Reflect.

## Sprint 1: Foundation (Days 1–5)

*Goal: Set up the development environment, provision Azure resources, implement authentication, establish the database schema, and scaffold the Flutter project. By the end of this sprint, a user can sign up, log in, and the backend can read/write to Cosmos DB. The Flutter app compiles and shows the auth flow.*

| # | Task | Owner | Priority | Est. Days | Status |
|---|---|---|---|---|---|
| 1.1 | Provision Azure resources (Cosmos DB, Container Apps, AD B2C, Blob Storage, Redis, AI Search, Content Safety) | Human | P0 | 2 | To Do |
| 1.2 | Initialize monorepo: backend (FastAPI) + flutter_app (Flutter), folder structure, linting, CI pipeline, gstack install | Claude | P0 | 0.5 | To Do |
| 1.3 | Implement Cosmos DB connection layer with tenant-aware database selection | Claude | P0 | 1 | To Do |

| | | | | | |
|---|---|---|---|---|---|
| 1.4 | Define all Pydantic models for Cosmos DB collections (users, workspaces, documents, knowledge_states, interactions, gamification, moderation_log, question_queue) | Claude | P0 | 1 | To Do |
| 1.5 | Configure Azure AD B2C tenant, user flows (signup, signin), social providers (Apple, Google) | Human | P0 | 1.5 | To Do |
| 1.6 | Implement auth middleware (JWT validation, user lookup, role extraction, Redis caching) | Claude | P0 | 1 | To Do |
| 1.7 | Build tenant and workspace CRUD API endpoints | Claude | P0 | 1 | To Do |
| 1.8 | Build user management API (direct creation + invite code system) | Claude | P0 | 1 | To Do |
| 1.9 | Write unit tests for auth middleware and CRUD endpoints (Boil the Lake: all 6 test types per endpoint) | Claude | P0 | 0.5 | To Do |
| 1.10 | Scaffold Flutter project: flavor system (admin/student), Riverpod DI, GoRouter, Dio client, theme, folder structure per feature-first arch | Claude | P0 | 1 | To Do |
| 1.11 | Build Flutter auth feature: login screen with Azure AD B2C SDK, token storage, auth state provider | Human | P0 | 1.5 | To Do |
| 1.12 | Build Flutter Dio networking layer with auth interceptor (token attach, refresh, error mapping) | Claude | P0 | 0.5 | To Do |
| 1.13 | Define shared Freezed models for User, Workspace, Tenant matching backend Pydantic schemas | Claude | P0 | 0.5 | To Do |

Definition of Done: Backend API running locally and on Azure Container Apps. A test user can register via AD B2C, receive a JWT, and call authenticated CRUD endpoints. Cosmos DB has tenant/workspace/user documents created successfully. Flutter app compiles in both flavors, shows login screen, authenticates via AD B2C, and can call backend endpoints. All backend endpoints have full test coverage.

## Sprint 2: Document Ingestion Pipeline (Days 6–10)

*Goal: Build the complete document upload and processing pipeline. By the end of this sprint, an admin can upload a PDF or image through the Flutter admin app, and the system extracts text, generates topics, chunks content, and stores everything in both Cosmos DB and Azure AI Search.*

| # | Task | Owner | Priority | Est. Days | Status |
|---|---|---|---|---|---|
| 2.1 | Set up Azure Document Intelligence resource and Python SDK integration | Human | P0 | 0.5 | To Do |
| 2.2 | Build document upload API endpoint (accept file, store in Blob Storage, queue for async processing) | Claude | P0 | 0.5 | To Do |
| 2.3 | Implement text extraction worker (PDF text extraction, OCR for images, Word/txt handling) | Both | P0 | 1.5 | To Do |
| 2.4 | Implement content safety scanning for uploaded documents (Azure Content Safety integration) | Claude | P0 | 1 | To Do |
| 2.5 | Build AI-powered topic extraction pipeline (document → structured topic list via AI model) | Both | P0 | 1.5 | To Do |
| 2.6 | Build taxonomy merge logic (new document topics merge with existing workspace taxonomy) | Claude | P0 | 1 | To Do |
| 2.7 | Build dependency graph inference (AI analyzes topics to determine prerequisites) | Both | P1 | 1 | To Do |
| 2.8 | Implement smart chunking (context-preserving, overlapping, flexible length) with metadata tagging | Claude | P0 | 1 | To Do |
| 2.9 | Push chunks to Azure AI Search with proper index schema and metadata filters | Both | P0 | 1 | To Do |
| 2.10 | Build document status polling API and processing status tracking | Claude | P1 | 0.5 | To Do |
| 2.11 | Build taxonomy CRUD API (get, update, regenerate) for admin editing | Claude | P1 | 0.5 | To Do |
| 2.12 | Build Flutter admin: document upload screen with file picker, upload progress, processing status indicator (all states: uploading, processing, ready, flagged, failed) | Human | P0 | 1.5 | To Do |
| 2.13 | Build Flutter admin: taxonomy viewer screen (visual topic tree from API) | Human | P1 | 1 | To Do |
| 2.14 | Write integration tests for the full ingestion pipeline | Claude | P1 | 0.5 | To Do |

Definition of Done: Admin uploads a PDF via the Flutter app, system extracts text, runs content safety check, extracts topics and builds taxonomy, chunks and vectorizes content into AI Search. Admin can see processing status in real-time and view the generated taxonomy. All data correctly stored in Cosmos DB. Document upload screen handles all states (Boil the Lake).

## Sprint 3: AI Question Generation & Adaptive Learning (Days 11–15)

*Goal: Build the adaptive learning engine and AI question generation pipeline. By the end of this sprint, a student can request a question through the Flutter app, and the system selects the right topic and difficulty, retrieves relevant content, and generates a personalized question via the AI model.*

| # | Task | Owner | Priority | Est. Days | Status |
|---|---|---|---|---|---|
| 3.1 | Set up Azure AI Foundry model deployment and Python SDK integration | Human | P0 | 1 | To Do |
| 3.2 | Build the MCP server skeleton (containerized Python service with tool definitions) | Claude | P0 | 1 | To Do |
| 3.3 | Implement Learning Path Engine Layer 1: topic selection algorithm (priority scoring based on mastery, recency, curriculum targets, dependency graph, variety factor) | Both | P0 | 2 | To Do |
| 3.4 | Implement difficulty calibration tool (target 70-75% success probability zone) | Claude | P0 | 0.5 | To Do |
| 3.5 | Build MCP tool: retrieve content from Azure AI Search with topic and metadata filters | Claude | P0 | 1 | To Do |
| 3.6 | Build MCP tool: retrieve student context from Cosmos DB (recent interactions, seen questions) | Claude | P0 | 0.5 | To Do |
| 3.7 | Design and implement question generation prompts (MCQ, long-answer, mathematical) with quality constraints. Run /prompt-eval on each. | Both | P0 | 1.5 | To Do |
| 3.8 | Build question validation and content safety check on AI-generated output | Claude | P0 | 0.5 | To Do |
| 3.9 | Build POST /questions/next endpoint (orchestrates full pipeline: select topic → calibrate difficulty → retrieve content → generate question → validate → return) | Both | P0 | 1 | To Do |
| 3.10 | Build POST /questions/{id}/answer endpoint (evaluate answer, update knowledge state, return feedback) | Claude | P0 | 1 | To Do |
| 3.11 | Implement knowledge state update logic (mastery score recalculation after each interaction) | Claude | P0 | 0.5 | To Do |

| | | | | | |
|---|---|---|---|---|---|
| 3.12 | Build flash card generation and rating endpoints | Claude | P1 | 1 | To Do |
| 3.13 | Build question prefetch worker (generate next question in background after student answers) | Claude | P2 | 0.5 | To Do |

Definition of Done: Student requests a question via API, system runs the adaptive learning algorithm to select topic and difficulty, retrieves relevant content, generates a quality question via AI, and returns it. Student submits an answer, system evaluates it, updates knowledge state, and returns feedback. Flash cards work similarly. End-to-end latency under 4 seconds for question generation. All prompts pass /prompt-eval with 5+ diverse inputs.

## Sprint 4: Flutter Core UI — Full Vertical Slices (Days 16–20)

*Goal: Build the core Flutter UI for both app flavors. Admin flavor gets workspace management, user management, and settings. Student flavor gets home page, question answering, flash cards, and progress view. Every screen handles all states (Boil the Lake). Every feature is a full vertical slice: backend data → repository → provider → screen → tests.*

| # | Task | Owner | Priority | Est. Days | Status |
|---|---|---|---|---|---|
| 4.1 | Build Flutter admin: workspace creation and management screens (create, list, edit, delete) with full state handling | Human | P0 | 1 | To Do |
| 4.2 | Build Flutter admin: user management (invite codes generation + display, student list, role assignment) with Riverpod providers | Human | P0 | 1 | To Do |
| 4.3 | Build Flutter admin: taxonomy editor (visual topic tree, drag-to-reorder, edit dependencies) — complex interactive widget | Human | P1 | 1.5 | To Do |
| 4.4 | Build Flutter admin: settings screen (question frequency slider, moderation toggles, gamification config, leaderboard toggle) | Human | P1 | 1 | To Do |
| 4.5 | Build Flutter admin: moderation dashboard (flagged content list, approve/reject actions, audit log) | Human | P1 | 1 | To Do |
| 4.6 | Build Flutter student: home page with daily tip cards, streak indicator, quick review button, topic progress rings | Human | P0 | 1.5 | To Do |
| 4.7 | Build Flutter student: question answering interface (MCQ with tap selection, long-answer with text input, mathematical input with notation) | Human | P0 | 2 | To Do |

| | | | | | |
|---|---|---|---|---|---|
| 4.8 | Build Flutter student: answer feedback screen (correct/incorrect animation, explanation display, XP earned, next question CTA) | Human | P0 | 1 | To Do |
| 4.9 | Build Flutter student: flash card interface (swipe-based review, flip animation, self-rating: easy/medium/hard) | Human | P0 | 1.5 | To Do |
| 4.10 | Build Flutter student: revision mode screen (mixed questions and flash cards, session progress indicator) | Human | P1 | 1 | To Do |
| 4.11 | Build Flutter student: progress view (per-topic mastery bars, overall level, recent activity timeline) | Human | P1 | 1 | To Do |
| 4.12 | Build all Riverpod providers, repositories, and Freezed models for question, flashcard, progress, and admin features | Claude | P0 | 2 | To Do |
| 4.13 | Write widget tests for all screens (all 4 states per screen: loading, error, empty, success) | Claude | P0 | 1.5 | To Do |

Definition of Done: Admin can log in (admin flavor), create a workspace, upload a document (from Sprint 2), view the generated taxonomy, manage users with invite codes, and configure workspace settings. Student can log in (student flavor), see their home page, answer AI-generated questions with feedback, review flash cards with swipe, enter revision mode, and view progress. All screens handle loading/error/empty states. Widget tests pass for all screens.

## Sprint 5: Gamification, Notifications & Analytics (Days 21–25)

*Goal: Implement the gamification system, push notifications for question delivery, and analytics dashboards for admins. The app should now feel engaging and provide meaningful progress feedback. This is where the app goes from functional to delightful.*

| # | Task | Owner | Priority | Est. Days | Status |
|---|---|---|---|---|---|
| 5.1 | Implement gamification engine backend (XP calculation, level progression, streak tracking, badge trigger evaluation) | Claude | P0 | 1 | To Do |
| 5.2 | Build gamification API endpoints (profile, leaderboard, badges, streak) | Claude | P0 | 1 | To Do |
| 5.3 | Design and implement badge/achievement system (definitions, trigger conditions, award logic) — at least 15 badges for launch | Both | P1 | 1 | To Do |

| | | | | | |
|---|---|---|---|---|---|
| 5.4 | Build Flutter student: gamification UI (XP counter with animated increment, level progress bar, streak flame icon with pulse animation, badge showcase grid, workspace leaderboard) | Human | P0 | 2 | To Do |
| 5.5 | Build Flutter: XP earned celebration animation (confetti particles on level-up, badge unlock modal with haptic feedback) | Human | P1 | 1 | To Do |
| 5.6 | Set up Firebase Cloud Messaging + Azure Notification Hubs for cross-platform push | Human | P0 | 1 | To Do |
| 5.7 | Build notification scheduling service (question frequency, streak reminders, milestone alerts, unanswered queue re-prompts) | Claude | P0 | 1 | To Do |
| 5.8 | Implement push notification handling in Flutter (foreground: in-app banner, background: system notification, tap: navigate to question) | Human | P0 | 1 | To Do |
| 5.9 | Build progress and analytics API endpoints (student progress, workspace analytics, tenant analytics) | Claude | P1 | 1 | To Do |
| 5.10 | Build Flutter admin: student progress detail view (per-student mastery chart, activity timeline, weak areas callout) | Human | P1 | 1 | To Do |
| 5.11 | Build Flutter admin: workspace analytics dashboard (avg mastery, engagement heatmap, topic difficulty distribution) | Human | P1 | 1 | To Do |
| 5.12 | Build unanswered questions queue logic (rejected questions re-queued for later, ask-again-later scheduling) | Claude | P2 | 0.5 | To Do |
| 5.13 | Write unit tests for gamification engine and notification scheduling | Claude | P0 | 0.5 | To Do |

Definition of Done: Students earn XP for every interaction, see animated level and streak on home page, view workspace leaderboards, and earn badges with celebratory animations. Push notifications deliver questions at admin-configured intervals across both iOS and Android. Admins can view per-student progress and workspace-level analytics. All gamification backend logic has full test coverage.

## Sprint 6: Polish, Testing & Launch Prep (Days 26–30)

*Goal: End-to-end testing, performance optimization, bug fixes, COPPA compliance review, app store submission preparation, and final polish. The app should be ready for a limited beta launch. Run /review, /qa, and /cso from gstack before declaring done.*

| # | Task | Owner | Priority | Est. Days | Status |
|---|---|---|---|---|---|
| 6.1 | End-to-end testing: full user journeys for family use case (parent creates tenant, adds child, uploads doc, child answers questions, parent views progress) | Both | P0 | 1 | To Do |
| 6.2 | End-to-end testing: full user journeys for school use case (teacher creates workspace, generates invite code, student joins, teacher monitors class progress) | Both | P0 | 1 | To Do |
| 6.3 | Performance optimization: question generation latency (<3s target), API response times, Redis cache hit rates, Flutter frame budget (no jank on 60fps) | Both | P0 | 1 | To Do |
| 6.4 | COPPA compliance review: age-gating flows, parental consent, data handling for minors, privacy policy | Human | P0 | 1 | To Do |
| 6.5 | Run /cso (gstack security audit): OWASP Top 10 + STRIDE threat model on all API endpoints, auth flows, and data paths | Claude | P0 | 1 | To Do |
| 6.6 | Run /review (gstack code review): full diff review, auto-fix obvious issues, flag completeness gaps | Claude | P0 | 0.5 | To Do |
| 6.7 | Build API versioning middleware (v1 prefix, version negotiation) | Claude | P1 | 0.5 | To Do |
| 6.8 | Implement offline question support in Flutter (cache current + next question locally, sync answers on reconnect via background isolate) | Human | P1 | 1.5 | To Do |
| 6.9 | Build onboarding flow in Flutter (first-time wizard: "Are you a parent or teacher?" → flavor-appropriate setup) | Human | P1 | 1 | To Do |
| 6.10 | Write API documentation (OpenAPI/Swagger spec auto-generated from FastAPI) | Claude | P1 | 0.5 | To Do |
| 6.11 | Set up monitoring and alerting (Azure Monitor, Application Insights, Crashlytics for Flutter) | Human | P1 | 1 | To Do |
| 6.12 | App Store + Play Store metadata preparation (screenshots, descriptions, privacy policy, age rating, data safety form) | Human | P0 | 1 | To Do |
| 6.13 | Bug fixes and UI polish based on testing feedback | Both | P0 | 2 | To Do |

| | | | | | |
|---|---|---|---|---|---|
| 6.14 | Prompt tuning: review and optimize all AI prompts based on generated question quality across subjects | Both | P1 | 1 | To Do |
| 6.15 | Run /retro (gstack retrospective): what shipped, what didn't, what to improve for post-launch sprints | Both | P1 | 0.5 | To Do |

Definition of Done: Both app flavors pass end-to-end testing for family and school journeys on iOS and Android. Question generation latency consistently under 3 seconds. All P0 bugs fixed. COPPA compliance reviewed and addressed. gstack /cso security audit passes with no CRITICAL findings. App Store and Play Store submissions ready. Monitoring and alerting configured. API documentation published. Sprint retrospective completed.

# 5. API Endpoint Quick Reference

All endpoints are prefixed with /api/v1. Authentication is required for all endpoints except /auth/register-completion and /auth/invite-code/validate.

## 5.1 Authentication

- POST /auth/register-completion — Post-signup profile creation, invite code redemption
- POST /auth/invite-code/validate — Check invite code validity
- GET /auth/me — Current user profile with roles and workspaces

## 5.2 Tenant & Workspace Management

- POST /tenants — Create tenant (admin signup)
- GET /tenants/{tenantId} — Tenant details
- POST /tenants/{tenantId}/workspaces — Create workspace
- GET /tenants/{tenantId}/workspaces — List workspaces
- PUT /workspaces/{workspaceId}/settings — Update workspace config
- POST /workspaces/{workspaceId}/invite-codes — Generate invite code

## 5.3 User Management

- POST /workspaces/{workspaceId}/users — Add user (direct or invite code)
- GET /workspaces/{workspaceId}/users — List workspace users
- PUT /workspaces/{workspaceId}/users/{userId}/role — Change role
- DELETE /workspaces/{workspaceId}/users/{userId} — Remove user (soft)

### 5.4 Documents & Taxonomy

- POST /workspaces/{workspaceId}/documents/upload — Upload document
- GET /workspaces/{workspaceId}/documents — List documents
- GET /workspaces/{workspaceId}/documents/{docId} — Document details
- GET /workspaces/{workspaceId}/documents/{docId}/status — Processing status
- DELETE /workspaces/{workspaceId}/documents/{docId} — Soft-delete
- GET /workspaces/{workspaceId}/taxonomy — Get topic tree
- PUT /workspaces/{workspaceId}/taxonomy — Update taxonomy
- POST /workspaces/{workspaceId}/taxonomy/regenerate — Rebuild taxonomy

### 5.5 Learning & Questions

- POST /workspaces/{workspaceId}/questions/next — Get next adaptive question
- POST /workspaces/{workspaceId}/questions/{qId}/answer — Submit answer
- GET /workspaces/{workspaceId}/flashcards/next — Get next flash card
- POST /workspaces/{workspaceId}/flashcards/{fcId}/rate — Rate flash card recall

### 5.6 Gamification

- GET /workspaces/{workspaceId}/users/{userId}/gamification — Gamification profile
- GET /workspaces/{workspaceId}/leaderboard — Workspace leaderboard
- GET /workspaces/{workspaceId}/users/{userId}/badges — Badges
- GET /workspaces/{workspaceId}/users/{userId}/streak — Streak details

### 5.7 Analytics & Progress

- GET /workspaces/{workspaceId}/users/{userId}/progress — Student progress
- GET /workspaces/{workspaceId}/analytics — Workspace analytics
- GET /tenants/{tenantId}/analytics — Tenant analytics

### 5.8 Moderation

- GET /workspaces/{workspaceId}/moderation/flagged — Flagged content
- PUT /workspaces/{workspaceId}/moderation/{itemId}/resolve — Resolve flagged item
- GET /workspaces/{workspaceId}/moderation/log — Audit trail

## 6. Risk Register

**Dynamic Taxonomy Quality (HIGH)**

AI-generated topic extraction and dependency graphs may be inaccurate, leading to poor learning paths. Mitigation: Build admin override tools, start with flat topic lists in v1, add hierarchy and dependencies iteratively. Monitor taxonomy quality through admin feedback. Search Before Building: check if existing taxonomy extraction libraries (e.g., educational ontologies) can seed the AI output.

**Question Generation Latency (MEDIUM)**

The full pipeline (topic selection, retrieval, AI generation, validation) may exceed the 3-second target. Mitigation: Implement question prefetching, cache hot data in Redis, optimize AI prompts for speed, consider using a faster model for simpler question types. Boil the Lake: build the prefetch system from Sprint 3, not as a Sprint 6 optimization afterthought.

**COPPA Compliance (HIGH)**

Serving minors in the US requires strict compliance with COPPA. Non-compliance carries significant legal risk. Mitigation: Implement age-gating in AD B2C, build parental consent flows, ensure no personal data collection from children under 13 without consent, legal review before launch.

**AI Content Safety (MEDIUM)**

AI-generated questions could occasionally produce inappropriate content despite safeguards. Mitigation: Two-tier moderation (prompt guardrails + output scanning), monitoring of regeneration rates, regular prompt tuning via /prompt-eval skill, human review of flagged content.

**Cross-Platform Flutter Parity (MEDIUM)**

Flutter renders consistently across iOS and Android, but platform-specific behaviors (push notifications, file picker, biometric auth, deep linking) may diverge. Mitigation: Test on both platforms from Sprint 1 onwards. Use platform channels only when necessary. Run /qa on both iOS simulator and Android emulator in Sprint 6.

**Scope Creep (MEDIUM)**

The feature set is ambitious for a 30-day MVP. Mitigation: Strict P0/P1/P2 prioritization. P2 tasks are explicitly deferrable. Sprint reviews gate progression. If a sprint falls behind, P2 tasks get cut before the timeline extends. The gstack /retro skill at the end of each sprint enforces honest assessment of what shipped vs. what didn't.

# 7. gstack Workflow Integration

Every sprint follows the gstack process: Think → Plan → Build → Review → Test → Ship → Reflect. Here is how the gstack skills map to our sprint activities.

### Sprint Start

/office-hours at the beginning of each sprint to pressure-test the sprint goal. Six forcing questions that reframe assumptions. /plan-eng-review to lock architecture decisions before coding starts. /plan-design-review for any new UI screens.

### During Development

/careful when working near auth, database, or production configuration. /freeze when debugging a specific module to prevent accidental edits elsewhere. /investigate for systematic root-cause debugging (no fixes without investigation). The Confusion Protocol: stop and ask when architecture is ambiguous.

### Before Merging

/review on every branch before merge. The code-reviewer agent enforces anti-slop standards. Auto-fixes obvious issues, flags completeness gaps. /cso for security-sensitive changes (auth, data access, content moderation).

### Before Release

/qa on the staging build (both iOS and Android). /ship to run tests, audit coverage, and open the PR. /document-release to update all project documentation to match what shipped.

### Sprint End

/retro to capture what shipped, what didn't, patterns to keep, and patterns to change. Learnings feed back into CLAUDE.md and rule files for the next sprint.